

# Guide de bonnes pratiques SQL

## ec28c97

Sébastien Nedjar - Mickaël Martin Nevot



Cette œuvre de Sébastien Nedjar est mise à disposition sous licence Creative Commons Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : [www.mickael-martin-nevot.com](http://www.mickael-martin-nevot.com)

---

## 1 Introduction

Ce guide présente les bonnes pratiques de rédaction SQL enseignées à l'IUT d'Aix-Marseille. L'objectif est double :

- **Lisibilité** : une requête bien formatée se comprend, se corrige et se maintient facilement ;
- **Fiabilité** : certaines habitudes préviennent des erreurs subtiles liées à NULL, aux doublons ou aux ambiguïtés.

Les exemples s'appuient sur la base de données Gestion pédagogique (R2.06) et la base Airbase (R1.05).

Un outil de vérification automatique, **SQLFluff**, est disponible pour contrôler le respect de ces règles (cf. section dédiée en fin de document).

## 2 Structure et mise en forme

### 2.1 Un mot-clé majeur par ligne

Chaque clause SQL principale doit commencer sur sa propre ligne. Cela permet de repérer immédiatement la structure logique de la requête.

À éviter :

```
SELECT nomEt, prenomEt FROM Etudiant WHERE anneeEt = 2 ORDER BY nomEt;
```

À adopter :

```
SELECT nomEt, prenomEt
FROM Etudiant
WHERE anneeEt = 2
ORDER BY nomEt;
```

Les clauses concernées sont : SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, UNION, INTERSECT, EXCEPT.

## 2.2 Indentation

L'indentation utilise **4 espaces** (jamais de tabulations). Elle s'applique dans les cas suivants.

**Conditions composées** — quand une clause (WHERE, ON, HAVING) comporte une **condition unique**, celle-ci reste sur la même ligne que le mot-clé. Quand la clause comporte **plusieurs conditions** (AND, OR), le mot-clé reste seul sur sa ligne et chaque condition est indentée d'un cran :

Condition unique :

```
SELECT nomEt, prenomEt
FROM Etudiant
WHERE anneeEt = 2;
```

Conditions composées :

```
SELECT nomEt, prenomEt
FROM Etudiant
WHERE
    anneeEt = 2
    AND groupeEt = 3;
```

Cette règle s'applique de la même manière au ON des jointures (cf. ci-dessous) et au HAVING.

**Jointures** — les JOIN sont indentés par rapport au FROM et le ON est indenté par rapport au JOIN. Pour les conditions du ON, la même règle de condition unique / composée s'applique :

Condition unique :

```
SELECT Et.numEt, nomEt, code
FROM Etudiant Et
    INNER JOIN Enseigne E
        ON Et.numEt = E.numEt;
```

Conditions composées :

```
SELECT Et.numEt, Et.nomEt, E.code
FROM Etudiant Et
    INNER JOIN Enseigne E
        ON
            Et.numEt = E.numEt
            AND E.code = 'BD';
```

**Sous-requêtes** — chaque niveau de sous-requête est indenté d'un cran supplémentaire :

```
SELECT numEt, nomEt
FROM Etudiant
WHERE numEt IN (
    SELECT numEt
    FROM Enseigne
    WHERE numProf IN (
```

```

        SELECT numProf
        FROM Professeur
        WHERE nomProf = 'BOITARD'
    )
);

```

**CTE (Common Table Expressions)** — le corps du WITH est indenté et suivi d’une ligne vide avant le SELECT principal :

```

WITH T (numProf, nbCode) AS (
    SELECT numProf, COUNT(DISTINCT code)
    FROM Enseigne
    GROUP BY numProf
)

SELECT numProf, nbCode
FROM T
WHERE nbCode > 3;

```

## 2.3 Mots-clés en majuscules

Les mots-clés SQL et les noms de fonctions s’écrivent en **lettres capitales**. Cela les distingue visuellement des noms de tables et de colonnes.

```

SELECT COUNT(DISTINCT numProf) AS nbProfs
FROM Enseigne E
    INNER JOIN Etudiant Et
        ON E.numEt = Et.numEt
WHERE anneeEt = 2
GROUP BY code
HAVING COUNT(DISTINCT numProf) > 2;

```

Les littéraux booléens et spéciaux suivent la même convention : NULL, TRUE, FALSE.

## 2.4 Point-virgule

Chaque requête se termine par un **point-virgule (;)**. C’est le délimiteur standard de fin d’instruction SQL.

## 2.5 Pas d’espace avant la parenthèse des fonctions

Les fonctions SQL sont immédiatement suivies de leur parenthèse ouvrante, sans espace.

À éviter :

```

SELECT COUNT (*)
FROM Professeur;

```

À adopter :

```

SELECT COUNT(*)
FROM Professeur;

```

## 2.6 Espaces et lignes blanches

- Pas d’espaces en fin de ligne ;
- une ligne vide entre chaque requête ;
- une ligne vide après la parenthèse fermante d’un CTE, avant le **SELECT** principal.

## 3 Conventions de nommage

L’enseignement adopte les conventions suivantes.

| Élément              | Convention                       | Exemples                                         |
|----------------------|----------------------------------|--------------------------------------------------|
| Mots-clés SQL        | MAJUSCULES ( <i>upper case</i> ) | <b>SELECT</b> , <b>WHERE</b> , <b>INNER JOIN</b> |
| Tables (relations)   | PascalCase                       | Etudiant, Module, OptionV                        |
| Attributs (colonnes) | camelCase                        | numEt, moyTest, coefCC, villeArr                 |
| Valeurs textuelles   | MAJUSCULES sans diacritique      | 'ROCCHI', 'MARSEILLE', 'INFORMATIQUE'            |
| Domaines             | MAJUSCULES au format D_XXX       | D_NUMPIL, D_VILLE                                |

Ces conventions permettent de distinguer immédiatement la nature de chaque élément dans une requête :

```
SELECT nomEt, prenomEt
FROM Etudiant
WHERE
    villeEt = 'MARSEILLE'
    AND anneeEt = 2;
```

## 4 Qualification des colonnes

### 4.1 Quand qualifier

Quand une requête fait intervenir **plusieurs tables**, il est recommandé de préfixer chaque colonne par le nom (ou l’alias) de sa table d’origine.

À éviter :

```
SELECT numEt, nomEt, code
FROM Etudiant Et
    INNER JOIN Enseigne E
    ON Et.numEt = E.numEt;
```

Le problème : `numEt` dans le **SELECT** est ambigu (il existe dans les deux tables). Même quand `nomEt` n’est pas ambigu (il n’existe que dans `Etudiant`), le qualifier rend la requête plus claire.

À adopter :

```
SELECT Et.numEt, Et.nomEt, E.code
FROM Etudiant Et
    INNER JOIN Enseigne E
    ON Et.numEt = E.numEt;
```

## 4.2 Pourquoi qualifier systématiquement

- Cela évite les erreurs d’ambiguïté (ORA-00918 sous Oracle, ERROR sous PostgreSQL) ;
- cela **documente l’intention** : le lecteur sait immédiatement de quelle table provient chaque colonne ;
- cela **facilite la maintenance** : si une colonne de même nom est ajoutée à une table, la requête ne casse pas.

## 4.3 Requêtes mono-table

Pour une requête ne portant que sur une seule table, la qualification est facultative.

# 5 Alias

## 5.1 Alias de colonnes : utiliser AS

Quand une colonne calculée ou renommée reçoit un alias, utiliser le mot-clé **AS** explicitement.

À éviter :

```
SELECT AVG(moyTest) moyenne
FROM Note
WHERE code = 'PRL';
```

À adopter :

```
SELECT AVG(moyTest) AS moyenne
FROM Note
WHERE code = 'PRL';
```

Le mot-clé **AS** rend l’alias visuellement distinct de la colonne. Sans lui, il est facile de confondre un alias avec une faute de frappe.

## 5.2 Alias de tables

En Oracle, le mot-clé **AS** n’est **pas supporté** pour les alias de tables. On écrit directement :

```
SELECT Et.numEt, Et.nomEt
FROM Etudiant Et;
```

Choisir des alias **courts mais significatifs** :

| Table      | Alias conseillé |
|------------|-----------------|
| Etudiant   | Et              |
| Professeur | P               |
| Module     | M               |
| Enseigne   | E               |
| Note       | N               |
| Pilote     | P               |

| Table | Alias conseillé |
|-------|-----------------|
| Avion | A               |
| Vol   | V               |

Pour les auto-jointures, utiliser des alias descriptifs :

```
SELECT P1.numPil, P1.nomPil
FROM Pilote P1
     INNER JOIN Pilote P2
           ON
               P1.numPil = P2.numPil
           AND P1.numPil <> P2.numPil;
```

## 6 Jointures

### 6.1 Forme recommandée : INNER JOIN ... ON

La forme explicite avec `INNER JOIN` est la forme à privilégier. Elle sépare clairement la **condition de jointure** (dans le `ON`) de la **condition de sélection** (dans le `WHERE`).

```
SELECT Et.numEt, Et.nomEt
FROM Etudiant Et
     INNER JOIN Note N
           ON Et.numEt = N.numEt
WHERE N.moyTest >= 10;
```

Écrire `INNER JOIN` plutôt que simplement `JOIN` : le résultat est identique mais l'intention est explicite.

### 6.2 Les trois formes de jointure

L'enseignement présente trois formes de jointure pour une même requête. Il est important de toutes les connaître.

**Version algébrique** — utilise `INNER JOIN ... ON` :

```
SELECT DISTINCT Et.numEt, Et.nomEt
FROM Etudiant Et
     INNER JOIN Enseigne E
           ON Et.numEt = E.numEt
     INNER JOIN Professeur P
           ON E.numProf = P.numProf
WHERE P.nomProf = 'LAPORTE';
```

**Version imbriquée** — utilise des sous-requêtes `IN` :

```
SELECT DISTINCT Et.numEt, Et.nomEt
FROM Etudiant Et
WHERE Et.numEt IN (
```

```

SELECT E.numEt
FROM Enseigne E
WHERE E.numProf IN (
    SELECT P.numProf
    FROM Professeur P
    WHERE P.nomProf = 'LAPORTE'
)
);

```

**Version prédicative** — utilise le produit cartésien avec conditions dans le WHERE :

```

SELECT DISTINCT Et.numEt, Et.nomEt
FROM Etudiant Et, Enseigne E, Professeur P
WHERE
    Et.numEt = E.numEt
    AND E.numProf = P.numProf
    AND P.nomProf = 'LAPORTE';

```

La version algébrique est la plus lisible et la plus utilisée en pratique. La version prédicative est historique et reste courante dans la littérature.

### 6.3 Jointures externes

Utiliser LEFT OUTER JOIN (ou LEFT JOIN) pour conserver les lignes sans correspondance. Préférer LEFT à RIGHT pour la lisibilité (la table « principale » est toujours à gauche).

```

SELECT Et.numEt, Et.nomEt, N.moyTest
FROM Etudiant Et
    LEFT OUTER JOIN Note N
        ON Et.numEt = N.numEt;

```

## 7 Opérateur « différent de »

Le standard SQL définit l’opérateur <> pour exprimer « différent de ». Bien que != soit accepté par certains SGBD, utiliser systématiquement <> pour la portabilité.

```

SELECT nomPil
FROM Pilote
WHERE nomPil <> 'DURAND';

```

## 8 Pièges classiques

### 8.1 NULL et NOT IN

L’un des pièges les plus fréquents en SQL. Quand une sous-requête dans un NOT IN peut retourner des valeurs NULL, **le résultat est toujours vide**.

Exemple problématique — la colonne **resp** contient des NULL dans la table **Module** :

```

-- ATTENTION : cette requête ne retourne aucun résultat !
SELECT nomProf, prenomProf

```

```
FROM Professeur
WHERE numProf NOT IN (
    SELECT resp
    FROM Module
);
```

La raison : NOT IN évalue `numProf <> NULL` pour chaque NULL dans la sous-requête, ce qui donne UNKNOWN, et la ligne est rejetée.

**Trois solutions correctes :**

Ajouter IS NOT NULL dans la sous-requête :

```
SELECT nomProf, prenomProf
FROM Professeur
WHERE numProf NOT IN (
    SELECT resp
    FROM Module
    WHERE resp IS NOT NULL
);
```

Utiliser NOT EXISTS (insensible à NULL) :

```
SELECT nomProf, prenomProf
FROM Professeur P
WHERE NOT EXISTS (
    SELECT *
    FROM Module M
    WHERE M.resp = P.numProf
);
```

Utiliser LEFT OUTER JOIN :

```
SELECT nomProf, prenomProf
FROM Professeur P
    LEFT OUTER JOIN Module M
        ON P.numProf = M.resp
WHERE M.resp IS NULL;
```

## 8.2 NULL et les calculs

Les fonctions d'agrégation (AVG, SUM, etc.) **ignorent les NULL**. Pour les calculs horizontaux (sur une même ligne), une seule valeur NULL rend toute l'expression NULL.

Utiliser COALESCE pour remplacer les NULL par une valeur par défaut :

```
SELECT AVG(
    (COALESCE(moyCC, 0) * coefCC + COALESCE(moyTest, 0) * coefTest)
    / (coefCC + coefTest)
) AS moyenne
FROM Note N
    INNER JOIN Module M
        ON N.code = M.code
WHERE M.libelle = 'PROLOG';
```



### 8.3 DISTINCT : quand l'utiliser

DISTINCT est **nécessaire** quand la requête produit des doublons qu'on souhaite éliminer. C'est typiquement le cas quand :

- une jointure « multiplie » les lignes (un étudiant ayant plusieurs enseignements d'un même professeur) ;
- on projette des colonnes qui ne forment pas une clef.

DISTINCT est **inutile** quand :

- la projection inclut la clef primaire de la table principale ;
- la sous-requête dans un IN ou NOT IN ne produit pas de doublons par construction (clef primaire).

*-- DISTINCT nécessaire : un étudiant peut avoir plusieurs enseignements  
-- du même professeur → son groupeEt serait dupliqué*

```
SELECT DISTINCT groupeEt
FROM Etudiant Et
     INNER JOIN Enseigne E
           ON Et.numEt = E.numEt
WHERE anneeEt = 2;
```

*-- DISTINCT inutile : numEt est clef primaire de Note,  
-- la sous-requête ne produit pas de doublons de code*

```
SELECT code, libelle
FROM Module
WHERE code IN (
    SELECT code
    FROM Note
);
```

### 8.4 GROUP BY : les colonnes non agrégées

Dans un SELECT avec GROUP BY, toute colonne qui n'est pas dans une fonction d'agrégation doit apparaître dans le GROUP BY.

Incorrect :

```
-- ERREUR : nomEt n'est pas dans le GROUP BY
SELECT Et.numEt, nomEt, COUNT(*)
FROM Etudiant Et
     INNER JOIN Enseigne E
           ON Et.numEt = E.numEt
GROUP BY Et.numEt;
```

Correct :

```
SELECT Et.numEt, Et.nomEt, COUNT(*)
FROM Etudiant Et
     INNER JOIN Enseigne E
           ON Et.numEt = E.numEt
GROUP BY Et.numEt, Et.nomEt;
```

## 8.5 WHERE vs HAVING

- WHERE filtre **avant** le partitionnement (sur les lignes individuelles) ;
- HAVING filtre **après** le partitionnement (sur les résultats des agrégations).

```
-- WHERE : filtre les étudiants de 2e année AVANT le partitionnement
-- HAVING : ne garde que les groupes ayant plus de 5 étudiants APRÈS
SELECT groupeEt, COUNT(*) AS nbEtudiants
FROM Etudiant
WHERE anneeEt = 2
GROUP BY groupeEt
HAVING COUNT(*) > 5;
```

Règle simple : si la condition porte sur une colonne individuelle → WHERE ; si elle porte sur un résultat de COUNT, SUM, AVG, etc. → HAVING.

## 8.6 SELECT \*

Éviter SELECT \* dans les requêtes finales :

- le nombre et l'ordre des colonnes dépendent du schéma → la requête est fragile ;
- la lisibilité est réduite car on ne sait pas quelles colonnes sont retournées.

**Exception acceptée** : dans les sous-requêtes EXISTS, SELECT \* est idiomatique car seule l'existence d'une ligne compte, pas son contenu :

```
SELECT nomProf
FROM Professeur P
WHERE NOT EXISTS (
    SELECT *
    FROM Module M
    WHERE M.resp = P.numProf
);
```

## 9 Récapitulatif

| Règle                     | Exemple correct                                      |
|---------------------------|------------------------------------------------------|
| Un mot-clé par ligne      | SELECT / FROM / WHERE chacun sur sa ligne            |
| Indentation 4 espaces     | AND anneeEt = 2                                      |
| Mots-clés en MAJUSCULES   | SELECT, INNER JOIN, COUNT                            |
| Qualification multi-table | Et.numEt (pas numEt seul)                            |
| Alias explicite           | COUNT(*) AS total                                    |
| Jointure explicite        | INNER JOIN ... ON                                    |
| Différent de              | <> (pas !=)                                          |
| Fin de requête            | ;                                                    |
| NOT IN sûr                | Ajouter WHERE col IS NOT NULL ou utiliser NOT EXISTS |
| DISTINCT raisonné         | Uniquement quand nécessaire, avec justification      |
| GROUP BY complet          | Toutes les colonnes non agrégées du SELECT           |

| Règle | Exemple correct |
|-------|-----------------|
|-------|-----------------|

## 10 Vérification automatique avec SQLFluff

**SQLFluff** est un *linter* SQL qui vérifie automatiquement le respect des règles de mise en forme. Il est configuré pour ce projet dans le fichier `.sqlfluff` à la racine du dépôt.

### 10.1 Installation

```
pip install sqlfluff
```

### 10.2 Utilisation

Vérifier un fichier :

```
sqlfluff lint mon-fichier.sql
```

Corriger automatiquement les problèmes de mise en forme :

```
sqlfluff fix mon-fichier.sql
```

### 10.3 Règles activées

Les principales règles vérifiées automatiquement par SQLFluff dans ce projet sont :

| Code | Règle                    | Description                                                  |
|------|--------------------------|--------------------------------------------------------------|
| CP01 | Mots-clés en majuscules  | <b>SELECT</b> et non <b>select</b>                           |
| CP03 | Fonctions en majuscules  | <b>COUNT</b> et non <b>count</b>                             |
| LT01 | Espaces parasites        | Pas d'espaces en fin de ligne                                |
| LT02 | Indentation              | 4 espaces, jointures et sous-requêtes indentées              |
| LT06 | Fonctions et parenthèses | <b>COUNT(*)</b> et non <b>COUNT (*)</b>                      |
| LT08 | Ligne vide après CTE     | Ligne vide entre <b>)</b> du <b>WITH</b> et le <b>SELECT</b> |
| AL02 | Alias de colonnes        | <b>AS</b> obligatoire : <b>COUNT(*) AS total</b>             |
| AM05 | Jointure qualifiée       | <b>INNER JOIN</b> et non <b>JOIN</b> seul                    |
| CV01 | Opérateur différent      | <b>&lt;&gt;</b> et non <b>!=</b>                             |
| CV06 | Point-virgule            | Chaque requête se termine par <b>;</b>                       |

Certaines règles sont volontairement désactivées pour tenir compte du contexte pédagogique (jointures prédictives, **SELECT \*** dans les exercices, identifiants camelCase du schéma). Consulter le fichier `.sqlfluff` pour le détail.